

Report Structure

Abstract

Introduction and Research Question

Dataset Generation

Models

Methodology

How do we plan to do this? Testing. Benchmarking.

Results

What do we infer from our results?

Prompt: “Finetune my model to distinguish between sad, angry, and happy human faces.”

Epochs: 3

Learning rate: 0.0001

Challenges

Known limitations and challenges we faced in building the experiment.

Conclusion & Future Works

Extensions of this work

References

TODO

Integrate backend with frontend

Last stuffs:

Identify CNN architecture automatically

Results

Results

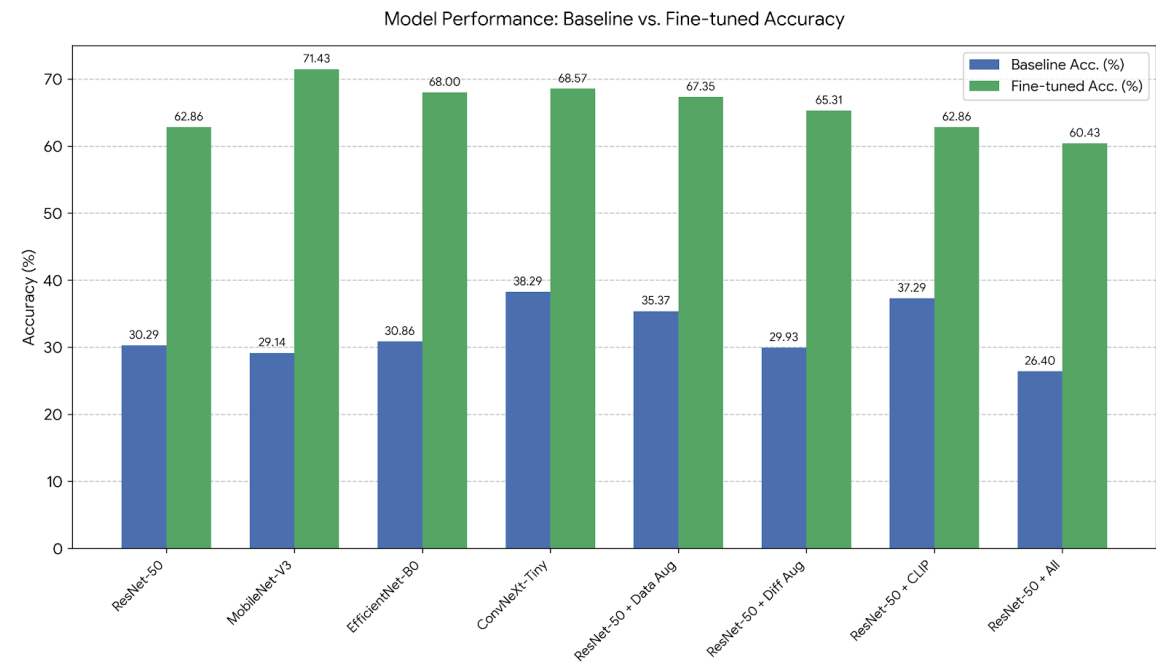
Experimental Setup

All runs used the following configuration:

Parameter	Value
Task	Distinguish between <i>sad</i> , <i>angry</i> , and <i>happy</i> human faces
Prompt	"Finetune my model to distinguish between sad, angry, and happy human faces."
Epochs	3
Learning Rate	0.0001

Eight experimental runs were conducted, varying the backbone architecture and data augmentation strategy. Each run reports baseline accuracy (ImageNet-pretrained head, no fine-tuning) and fine-tuned accuracy on the test set.

Summary of Results



	Run	Configuration	Baseline Acc. (%)	Fine-tuned Acc. (%)	Gain (pp)
	1	ResNet-50	30.29	62.86	+32.57
	2	MobileNet-V3	29.14	**71.43**	+42.29
	3	EfficientNet-B0	30.86	68.00	+37.14
	4	ConvNeXt-Tiny	**38.29**	68.57	+30.28
	5	ResNet-50 + Data Augmentation	35.37	67.35	+31.98
	6	ResNet-50 + Diffusion Augmentation	29.93	65.31	+35.38
	7	ResNet-50 + CLIP Filtering	37.29	62.86	+25.57
	8	ResNet-50 + All Features	26.40	60.43	+34.03

Fine-tuning consistently improved accuracy across all runs, with gains ranging from ****+25.6 to +42.3 percentage points****.

Per-Run Results

Run 1 – ResNet-50 (Baseline Architecture)

- ****Baseline accuracy:**** 30.29%
- ****Fine-tuned accuracy:**** 62.86%

Per-class accuracy chart – Baseline vs. Fine-tuned (ResNet-50)

> The model completely failed to detect the **sad** class before fine-tuning (0.0%), but recovered to 42.2% after. The **angry** and **happy** classes showed more modest gains (65.4% → 63.5% and 32.2% → 84.7%, respectively).

Run 2 – MobileNet-V3

- ****Baseline accuracy:**** 29.14%
- ****Fine-tuned accuracy:**** 71.43% **(highest among all runs)**

Per-class accuracy chart – Baseline vs. Fine-tuned (MobileNet-V3)

> MobileNet-V3 achieved the best overall fine-tuned accuracy, with strong and balanced per-class performance across all three emotion categories after fine-tuning.

Run 3 – EfficientNet-B0

- ****Baseline accuracy:**** 30.86%
- ****Fine-tuned accuracy:**** 68.00%

Per-class accuracy chart – Baseline vs. Fine-tuned (EfficientNet-B0)

> EfficientNet-B0 showed substantial gains across all three classes, with a notably low baseline on **angry** (13.5%) that improved to 65.4% after fine-tuning.

Run 4 – ConvNeXt-Tiny

- ****Baseline accuracy:**** 38.29% **(highest baseline among all runs)**
- ****Fine-tuned accuracy:**** 68.57%

Per-class accuracy chart – Baseline vs. Fine-tuned (ConvNeXt-Tiny)

> ConvNeXt-Tiny had the strongest out-of-the-box performance, reflecting its modern architectural priors. However, despite the head start, it did not surpass MobileNet-V3 after fine-tuning.

Run 5 – ResNet-50 with Standard Data Augmentation

- ****Baseline accuracy:**** 35.37%
- ****Fine-tuned accuracy:**** 67.35%

Per-class accuracy chart – Baseline vs. Fine-tuned (ResNet-50 + Augmentation)

> Standard data augmentation improved the ResNet-50 baseline by ~5 percentage points and lifted fine-tuned accuracy by ~4.5pp over the plain ResNet-50. Notably, the **happy** class baseline was 0.0%, suggesting augmentation shifted the class distribution.

Run 6 – ResNet-50 with Diffusion-Generated Image Augmentation

- ****Baseline accuracy:**** 29.93%
- ****Fine-tuned accuracy:**** 65.31%

**Per-class accuracy chart – Baseline vs. Fine-tuned (ResNet-50 + Diffusion) **

> Diffusion-generated augmentation produced a middle-ground result – outperforming the plain ResNet-50 fine-tuned accuracy (62.86%) but falling short of standard augmentation (67.35%), suggesting that synthetic images provide useful diversity but may introduce distributional noise.

Run 7 – ResNet-50 with CLIP-Based Filtering

- ****Baseline accuracy:**** 37.29%
- ****Fine-tuned accuracy:**** 62.86%

**Per-class accuracy chart – Baseline vs. Fine-tuned (ResNet-50 + CLIP) **

> CLIP filtering produced the second-highest baseline (37.29%), indicating that quality-filtered data improves zero-shot representations. However, this advantage did not transfer to fine-tuning – the fine-tuned accuracy matched the plain ResNet-50 (62.86%), the lowest among augmentation strategies. The **sad** class saw a significant regression (83.7% → 40.8%) post fine-tuning, warranting further investigation.

Run 8 – ResNet-50 with All Features Enabled

- **Baseline accuracy:** 26.40% **(lowest baseline among all runs)**
- **Fine-tuned accuracy:** 60.43% **(lowest among ResNet-50 variants)**

Per-class accuracy chart – Baseline vs. Fine-tuned (ResNet-50 + All Features)

> Combining all augmentation strategies simultaneously produced the worst performance among ResNet-50 variants in both baseline and fine-tuned accuracy. This indicates that naively stacking augmentation pipelines can interfere with training stability and introduce conflicting noise signals that hurt convergence.

Analysis and Discussion

Architecture Comparison (Runs 1-4)

Among the four backbone architectures tested without augmentation, **MobileNet-V3 achieved the highest fine-tuned accuracy (71.43%)**, outperforming ResNet-50, EfficientNet-B0, and ConvNeXt-Tiny. This suggests that lighter, depthwise-separable architectures generalize better to this facial emotion dataset, potentially because their inductive biases align well with local texture features relevant to emotion recognition.

ConvNeXt-Tiny stands out for having the best baseline (38.29%), reflecting stronger ImageNet-pretrained feature representations. Yet it did not surpass MobileNet-V3 after fine-tuning, implying that baseline feature quality and fine-tuning performance are not perfectly correlated for this task.

Effect of Augmentation Strategies on ResNet-50 (Runs 1, 5-8)

Strategy	Fine-tuned Acc. (%)	vs. Plain ResNet-50
None (Run 1)	62.86	-
Data Augmentation (Run 5)	67.35	+4.49
Diffusion Augmentation (Run 6)	65.31	+2.45
CLIP Filtering (Run 7)	62.86	+0.00
All Combined (Run 8)	60.43	-2.43

Standard data augmentation offered the most consistent benefit. Diffusion-generated images provided a modest gain, while CLIP filtering raised the baseline but did not improve fine-tuned accuracy. Combining all strategies hurt performance, suggesting diminishing or negative returns from stacking augmentation methods without careful tuning.

Key Takeaways

1. ****Fine-tuning is highly effective:**** All architectures showed 22-42 pp gains over baseline, confirming that task-specific fine-tuning is essential for facial emotion recognition.
2. ****Lighter architectures can outperform heavier ones:**** MobileNet-V3 surpassed ResNet-50 and ConvNeXt-Tiny after fine-tuning despite having fewer parameters.
3. ****Strong baselines do not guarantee strong fine-tuning outcomes:**** ConvNeXt-Tiny and CLIP-filtered ResNet-50 had the best baselines but did not achieve the best fine-tuned accuracy.
4. ****Augmentation stacking requires care:**** Naively combining data augmentation, diffusion generation, and CLIP filtering degraded both baseline and fine-tuned accuracy (Run 8), highlighting the need for principled augmentation selection.

Limitations & Challenges

Limitations and Challenges

While the pipeline produced meaningful improvements across all runs, several limitations were observed. Training results exhibited inconsistency across runs, likely due to the subtle and overlapping visual features that distinguish facial emotion classes — a challenge that may require careful, manual curation of the dataset to achieve more reliable performance. Additionally, because the training data is sourced externally, it may contain malicious or low-quality content; to mitigate this, the pipeline includes a filtering step to remove such samples before training. Finally, training time scales linearly with the number of target classes, which may become a practical bottleneck as the classification task is expanded beyond the three emotions evaluated here.

Conclusion & Future Scope

This project demonstrates that combining CLIP-based filtering with LLM orchestration creates a viable, fully automated pipeline for zero-shot data collection and model fine-tuning. Across all eight experimental runs, fine-tuning consistently delivered accuracy gains of +25.6 to +42.3 percentage points without any manually labeled data, with MobileNet-V3 achieving the best fine-tuned accuracy at 71.43%. Despite these results, training outcomes were inconsistent across runs due to the subtle visual overlap between emotion classes, external data sources introduced quality risks mitigated only partially by filtering, and training time scales linearly with the number of classes — all challenges that limit the pipeline's robustness at scale. Future work will focus on three key extensions: supporting user-uploaded images for hybrid datasets, integrating GPT-4V for automated label verification and active learning, and expanding beyond image classification to object detection frameworks such as YOLO — moving the system from a research prototype toward a general-purpose tool for prompt-driven model development.

Data Generation and Processing

Data Generation and Processing

The data pipeline is composed of four discrete, sequentially orchestrated subsystems: prompt parsing, image acquisition (web scraping and synthetic generation), quality filtering, and augmentation. Each stage is triggered by the central FastAPI orchestrator in `src/main.py`.

Sourcing

LLM-Driven Prompt Parsing

Raw user input (e.g., "Fine-tune this to detect healthy vs. diseased tomato leaves") is routed to the orchestrator module (`src/orchestrator/orchestrator.py`), which delegates to a large language model via the Groq inference API. The LLM is instructed, via a structured system prompt, to return a JSON object containing (a) the extracted class labels (singularized) and (b) three highly specific Google Image search queries per class. The response is sanitized of any markdown fencing and repaired using the `json_repair` library before deserialization. The system prompt is defined as follows:

```
# src/orchestrator/orchestrator.py

system_prompt = """
You are an AI assistant helping to build computer vision datasets.
The user will provide a description of a vision classifier they
want to train. You must extract the exact target classes and
generate 3 highly specific Google Image search queries for each
class.

You MUST respond ONLY with a valid JSON object in this exact format:
{
  "classes": ["class1", "class2"],
  "search_queries": {
    "class1": ["query 1", "query 2", "query 3"],
    "class2": ["query 1", "query 2", "query 3"]
  }
}

The target classes should be made singular while generating the
```

```
JSON object. Make sure that the search queries are highly relevant
with the corresponding class.
"""
```

Web Scraping (Bing Images)

For each class-keyword pair, the scraper module (`src/scraper/scraper.py`) launches a headless Chromium browser via Playwright, navigates to Bing Image Search, waits for JavaScript-rendered tile anchors (`a.iusc`), scrolls to trigger lazy-loading, and then extracts full-resolution image URLs from the `murl` field embedded in each tile's JSON `m` attribute. BeautifulSoup handles the HTML parsing. A fallback path extracts thumbnail `src` attributes from `img.mimg` elements when the primary selector yields no results. The scraper is configurable via environment variables: `MAX_IMAGES` (default 20 per query), `REQUEST_TIMEOUT` (default 30s), and a custom `USER_AGENT` string.

Synthetic Image Generation (Diffusion Models)

When the user enables synthetic generation, the system invokes `generate_diffusion_dataset()` in `src/diffusion/flux_api_hf.py`. This function first calls the LLM (`openai/gpt-oss-120b` via Groq) to produce an optimized base prompt for the given class, then requests up to 100 prompt variations through a second LLM call (`prompt_variations()`). Each variation is sent to the Hugging Face Inference API targeting the `FLUX.1-schnell` model (`black-forest-labs/FLUX.1-schnell`). Generated images are saved as PNG files with a SHA-256-derived run hash in the filename for deduplication.

Pipeline and Preprocessing

Train/Test Splitting

The `DataGenerator` class (`src/data_generator/main.py`) reorganizes the flat `downloads/<class>/` directory structure into `downloads/train/<class>/` and `downloads/test/<class>/` subdirectories using a seeded (`seed=42`) 80/20 random split. At least one test image is guaranteed per class when more than one image is available. The split is idempotent: if the `train/` and `test/` directories already exist, it short-circuits with existing counts.

Data Augmentation

Applied exclusively to the training split, augmentation generates `N` copies per original image (default 5, configurable via `copies_per_image`) using a `torchvision.transforms.Compose` pipeline. The augmentation transforms are defined as follows:

```
# src/data_generator/main.py

AUGMENTATION_PIPELINE = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=20),
    transforms.RandomResizedCrop(224, scale=(0.7, 1.0)),
    transforms.ColorJitter(
        brightness=0.3,
        contrast=0.3,
        saturation=0.3,
        hue=0.1
    ),
])
```

Augmented files are prefixed with "aug_" to prevent re-augmentation on subsequent runs.

CLIP Relevance Filtering

When enabled, the CLIPRelevanceVerifier (src/filters/clip_relevance.py) loads the openai/clip-vit-base-patch32 model and computes cosine similarity between each image's CLIP embedding and a text probe constructed from the class label and user prompt (via make_text_probe() in src/filters/text_probes.py). Images scoring below RELEVANCE_THRESHOLD (default 0.22) are deleted from disk. The filter is applied to both train/ and test/ subdirectories for each class. A minimum post-filter image count (MIN_IMAGES_AFTER_FILTER, default 3) is enforced; if any class falls below this threshold, the pipeline returns an error rather than training on an insufficient dataset.

Corrupt Image Removal

During data loading, the Trainer.remove_corrupt_images() method attempts to open and fully load every image with Pillow. Files that fail are deleted from disk and excluded from the dataset.

Normalization for Model Ingestion

At training and evaluation time, all images are resized to 224×224 pixels and normalized to ImageNet channel statistics. The preprocessing transform chain is:

```
# src/trainer/train.py

data_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])
```

Storage and Formatting

Local Filesystem

Images are stored on disk under `downloads/<class_name>/` (post-scraping) and reorganized into `downloads/train/<class>/` and `downloads/test/<class>/` after splitting. Filenames include a zero-padded index and an 8-character MD5 hash of the source URL for deduplication. The `LocalStorage` class (`src/scrapper/storage.py`) handles streaming downloads with configurable timeouts.

SQLite Metadata Store

An optional SQLite database (`image_scraper.db`) persists lightweight metadata per image. The schema enforces a `UNIQUE(keyword, image_url)` constraint with `INSERT OR IGNORE` semantics to handle duplicate URLs gracefully. Image binary data is never stored in the database. The schema is as follows:

Column	Type	Constraint
id	INTEGER	PRIMARY KEY AUTOINCREMENT
keyword	TEXT	NOT NULL
image_url	TEXT	NOT NULL
local_path	TEXT	—
scraped_at	TEXT	NOT NULL

Table 1. SQLite images table schema.

PyTorch Dataset

For training, `torchvision.datasets.ImageFolder` is used to infer class labels from the directory hierarchy under `downloads/train/`. The trainer performs label remapping to ensure contiguous

0-based indices compatible with `nn.CrossEntropyLoss`, filtering samples to include only the classes specified in the original user prompt.

Methodology

System Architecture

The system follows a three-tier architecture: a React single-page application (frontend), a FastAPI server (backend orchestrator), and external inference APIs (Groq for LLM, Hugging Face for diffusion).

The end-to-end pipeline executes the following stages sequentially within a single synchronous POST `/run/` request handler:

- 1. Prompt Parsing** — The user's natural-language prompt is forwarded to the Groq-hosted LLM, which returns structured JSON containing class names and per-class search queries.
- 2. Data Collection** — For each class-keyword pair, the scraper launches a headless browser, retrieves image URLs from Bing, and downloads them to local disk.
- 3. Synthetic Generation (optional)** — The LLM generates diffusion-optimized prompt variations, which are sent to the FLUX.1-schnell model via the Hugging Face Inference API. Generated images are saved alongside scraped images.
- 4. Dataset Assembly** — The DataGenerator splits images into train/test sets (80/20).
- 5. CLIP Filtering (optional)** — Low-relevance images are purged from both splits based on CLIP cosine similarity scores.
- 6. Augmentation (optional)** — Training images are augmented with random geometric and photometric transforms.
- 7. Fine-Tuning** — A pretrained CNN backbone is loaded, its feature-extraction layers are frozen, and a new classification head is trained on the assembled dataset.
- 8. Evaluation** — Triggered separately via POST `/test/`, which loads the finetuned checkpoint and a fresh pretrained baseline, runs inference on the held-out test set, and returns confusion matrices and per-class accuracy metrics for both.

Algorithmic Choices

Transfer Learning with Frozen Backbones

The project adopts a transfer-learning paradigm: a CNN pretrained on ImageNet-1K is loaded, all backbone parameters are frozen (requires_grad = False), and only a newly initialized classification head (a single nn.Linear layer matching the number of target classes) is trained. This design choice is motivated by the small, noisy datasets typical of automated scraping and generation — freezing the backbone prevents catastrophic forgetting and reduces training time to minutes rather than hours.

CLIP-Based Quality Assurance

Rather than relying solely on keyword relevance from the search engine, the system employs CLIP (openai/clip-vit-base-patch32) as a semantic filter. By computing image–text cosine similarity against a probe derived from the class label and the user’s task description, the system removes off-topic or low-quality images before they enter the training set. The text probe is constructed as:

```
# src/filters/text_probes.py

def make_text_probe(class_name, user_prompt=None):
    if user_prompt:
        return f'{user_prompt}. A clear photo showing: {class_name}.'
    return f'a photo of {class_name}'
```

LLM-as-Orchestrator Pattern

The LLM serves a dual role: (a) as a structured information extractor (parsing natural language into class labels and search queries) and (b) as a prompt engineer (generating diverse, high-quality diffusion prompts). The json_repair library provides resilience against malformed JSON output from the LLM, a common failure mode with instruction-following models.

Headless Browser Scraping

Playwright was chosen over static HTTP scraping (e.g., requests + BeautifulSoup alone) because Bing Image Search renders results via JavaScript, making server-side HTML insufficient. The scraper performs three scroll iterations to trigger lazy-loading of additional image tiles.

Integration

Backend API (FastAPI)

The backend exposes four REST endpoints:

Endpoint	Method	Purpose
/run/	POST	Executes the full data-generation and training pipeline

/test/	POST	Runs baseline vs. finetuned evaluation on the test split
/parse-user-prompt/	POST	Standalone prompt-parsing (LLM call only)
/generate-dataset/	POST	Standalone scraping by keyword

Table 2. FastAPI REST endpoint inventory.

Request/response bodies use Pydantic models (RunRequest, DataSources, AdvancedSettings) for validation. CORS is configured to allow requests from localhost:5173 and localhost:3000 (Vite and alternative dev server ports).

Frontend (React + Vite + Tailwind CSS)

The SPA communicates with the backend via `fetch()` calls in `src/frontend/src/utlis/api.ts`. The primary interaction flow is:

1. The user fills out a PromptForm component (prompt, email, model selection, data source toggles, advanced hyperparameters).
2. On submission, a JSON payload is POSTed to `/run/`. The request is synchronous and blocking — the frontend displays a timed pipeline progress visualization (PipelineStepper) driven by estimated per-stage durations while waiting for the backend response.
3. After `/run/` returns, the frontend transitions to a polling phase, sending POST `/test/` requests every 5 seconds (with a configurable 75-second timeout) until evaluation metrics are available.
4. Results (confusion matrices, accuracy delta) are persisted in `localStorage` and rendered on a dedicated Results page.

Run state (phase, timestamps, errors, data-source configuration) is synchronized across pages via `localStorage` and a custom `runstatechange` DOM event, consumed by the `useRunState()` hook.

External API Integrations

Groq API: The CustomLLM class (`src/utlis/llm.py`) wraps a standard OpenAI-compatible chat completions endpoint. Authentication is via a bearer token (`GROQ_API_KEY`). The `trust_env` flag on the `requests.Session` is explicitly disabled to avoid proxy interference.

Hugging Face Inference API: The `InferenceClient` from `huggingface_hub` is used with the `hf-inference` provider for text-to-image generation. Authentication is via `HF_TOKEN`.

Models

Architecture and Selection

The system employs models across four distinct functional roles:

Large Language Model (Prompt Parsing and Prompt Engineering)

Attribute	Value
Model	openai/gpt-oss-120b (120B-parameter open-source model)
Provider	Groq inference API
Usage	Extracting class labels and search queries; generating base diffusion prompts; producing up to 100 prompt variations

Table 3. LLM specification.

Text-to-Image Diffusion Model (Synthetic Data Generation)

Attribute	Value
Primary Model	black-forest-labs/FLUX.1-schnell (fast distilled variant)
Provider	Hugging Face Inference API (serverless)
Explored Alternative 1	stable-diffusion-v1-5 — local inference via diffusers AutoPipelineForText2Image, FP16, torch.compile
Explored Alternative 2	stable-diffusion-xl-base-1.0 via StableDiffusionAPI REST endpoint
Explored Alternative 3	FLUX.1-dev via Gradio client

Table 4. Diffusion model specification and explored alternatives.

CLIP Model (Relevance Filtering)

Attribute	Value
Model	openai/clip-vit-base-patch32 (ViT-B/32)
Provider	Loaded locally via transformers.CLIPModel and CLIPProcessor
Usage	Cosine similarity between image and text embeddings to filter irrelevant scraped images

Table 5. CLIP model specification.

Vision Classification Models (Fine-Tuning Targets)

The user selects from four ImageNet-1K pretrained architectures, all loaded via torchvision.models:

Enum Value	Architecture	Pretrained Weights
RESNET_50	ResNet-50	ResNet50_Weights.IMAGENET1K_V1
MOBILENET_V3	MobileNetV3-Large	MobileNet_V3_Large_Weights.IMAGENET1K_V1
EFFICIENTNET	EfficientNet-B0	EfficientNet_B0_Weights.IMAGENET1K_V1
CONV_NEXT	ConvNeXt-Tiny	ConvNeXt_Tiny_Weights.IMAGENET1K_V1

Table 6. Supported vision model architectures and their pretrained weight identifiers.

Implementation Details

LLM Access

The CustomLLM wrapper (`src/utils/llm.py`) constructs a standard OpenAI-compatible chat completions payload and sends it to the configured Groq endpoint. No fine-tuning or local model hosting is performed — all LLM inference is API-based.

Diffusion Model Access

FLUX.1-schnell is accessed exclusively via the Hugging Face serverless Inference API (`InferenceClient.text_to_image()`). No local diffusion model weights are downloaded or fine-tuned. The alternative Stable Diffusion v1.5 path in `text_to_image.py` uses `AutoPipelineForText2Image.from_pretrained()` with FP16 precision and `torch.compile(mode="reduce-overhead", fullgraph=True)` for UNet acceleration, but this path is not invoked by the main pipeline.

CLIP Model Access

The CLIP model is loaded locally from Hugging Face Hub on first use (lazy initialization) and cached in memory for subsequent scoring calls within the same process lifetime. Inference runs in `torch.no_grad()` mode with explicit L2-normalization of both image and text embeddings before computing dot-product similarity.

Vision Model Fine-Tuning

The ModelBuilder class (`src/utils/model_builder.py`) implements a transfer-learning strategy:

1. Load a pretrained backbone from `torchvision.models` with official ImageNet-1K weights.
2. Freeze all parameters: `param.requires_grad = False` for every parameter.
3. Replace the final classification head via `_replace_classifier_head()`, which locates the terminal `nn.Linear` layer (`.fc` for ResNet, or the last `nn.Linear` found via `named_modules()` for other architectures) and replaces it with a new `nn.Linear(in_features, num_classes)`. The new head's parameters are trainable by default.
4. Move the model to GPU if available.

The trained checkpoint is saved as a dictionary containing both the `state_dict` and the architecture identifier string, enabling the evaluation module to reconstruct the correct skeleton at load time. The classifier head replacement logic is:

```
# src/utils/model_builder.py

for param in model.parameters():
    param.requires_grad = False

# For ResNet family:
in_features = model.fc.in_features
model.fc = nn.Linear(in_features, num_classes)

# For other architectures: find last nn.Linear via named_modules()
# and replace with nn.Linear(in_features, num_classes)
```

Parameters and Prompting

Training Hyperparameters

Parameter	Default Value	Source
Optimizer	Adam	src/trainer/train.py
Learning rate	0.003	src/trainer/train.py
Loss function	nn.CrossEntropyLoss	src/trainer/train.py
Batch size (training)	50	src/trainer/train.py
Batch size (evaluation)	32	src/trainer/test.py
Epochs	12 (user-configurable)	src/utils/models.py
Input resolution	224 × 224	src/trainer/train.py
Train/test split	80 / 20	src/data_generator/main.py
Augmented copies per image	5	src/data_generator/main.py
Random seed (split)	42	src/data_generator/main.py

Table 7. Training hyperparameters with default values and source locations.

CLIP Filtering Parameters

Parameter	Default Value	Source
CLIP model	openai/clip-vit-base-patch32	src/config.py
Relevance threshold	0.22	src/config.py
Minimum images after filter	3	src/config.py

Table 8. CLIP relevance filtering configuration defaults.

Explored Diffusion Parameters

The following diffusion-related parameters were explored in experimental scripts but are not used in the main production pipeline:

Parameter	Value	Source
Guidance scale sweep	0.0 through 10.0 (step 1.0)	text_to_image.py
Precision	FP16 (torch.float16)	text_to_image.py
Inference steps (SD API)	30	stable_diffusion_api.py
Guidance scale (SD API)	7.5	stable_diffusion_api.py
Resolution (SD API)	512 × 512	stable_diffusion_api.py
Negative prompt	Defined to suppress common artifacts	stable_diffusion_api.py

Table 9. Explored diffusion model parameters from experimental scripts.

Markdown

Data Generation and Processing

The data pipeline is composed of four discrete, sequentially orchestrated subsystems: prompt parsing, image acquisition (web scraping and synthetic generation), quality filtering, and augmentation. Each stage is triggered by the central FastAPI orchestrator in `src/main.py`.

Sourcing

LLM-Driven Prompt Parsing

Raw user input (e.g., "Fine-tune this to detect healthy vs. diseased tomato leaves") is routed to the orchestrator module (`src/orchestrator/orchestrator.py`), which delegates to a large language model via the Groq inference API. The LLM is instructed, via a structured system prompt, to return a JSON object containing (a) the extracted class labels (singularized) and (b) three highly specific Google Image search queries per class. The response is sanitized of any markdown fencing and repaired using the `json\_repair` library before deserialization. The system prompt is defined as follows:

```
```python
src/orchestrator/orchestrator.py

system_prompt = """
You are an AI assistant helping to build computer vision datasets.
The user will provide a description of a vision classifier they
want to train. You must extract the exact target classes and
generate 3 highly specific Google Image search queries for each
class.

You MUST respond ONLY with a valid JSON object in this exact format:
{
 "classes": ["class1", "class2"],
 "search_queries": {
 "class1": ["query 1", "query 2", "query 3"],
 "class2": ["query 1", "query 2", "query 3"]
 }
}
The target classes should be made singular while generating the
JSON object. Make sure that the search queries are highly relevant
with the corresponding class.
"""
```

...

### ### Web Scraping (Bing Images)

For each class-keyword pair, the scraper module (`src/scraper/scraper.py`) launches a headless Chromium browser via Playwright, navigates to Bing Image Search, waits for JavaScript-rendered tile anchors (`a.iusc`), scrolls to trigger lazy-loading, and then extracts full-resolution image URLs from the `murl` field embedded in each tile's JSON `m` attribute. BeautifulSoup handles the HTML parsing. A fallback path extracts thumbnail `src` attributes from `img.mimg` elements when the primary selector yields no results. The scraper is configurable via environment variables: `MAX_IMAGES` (default 20 per query), `REQUEST_TIMEOUT` (default 30s), and a custom `USER_AGENT` string.

### ### Synthetic Image Generation (Diffusion Models)

When the user enables synthetic generation, the system invokes `generate_diffusion_dataset()` in `src/diffusion/flux_api_hf.py`. This function first calls the LLM (`openai/gpt-oss-120b` via Groq) to produce an optimized base prompt for the given class, then requests up to 100 prompt variations through a second LLM call (`prompt_variations()`). Each variation is sent to the Hugging Face Inference API targeting the FLUX.1-schnell model (`black-forest-labs/FLUX.1-schnell`). Generated images are saved as PNG files with a SHA-256-derived run hash in the filename for deduplication.

## ## Pipeline and Preprocessing

### ### Train/Test Splitting

The `DataGenerator` class (`src/data_generator/main.py`) reorganizes the flat `downloads/<class>/` directory structure into `downloads/train/<class>/` and `downloads/test/<class>/` subdirectories using a seeded (`seed=42`) 80/20 random split. At least one test image is guaranteed per class when more than one image is available. The split is idempotent: if the `train/` and `test/` directories already exist, it short-circuits with existing counts.

### ### Data Augmentation

Applied exclusively to the training split, augmentation generates `N` copies per original image (default 5, configurable via `copies_per_image`) using a `torchvision.transforms.Compose` pipeline. The augmentation transforms are defined as follows:

```

```python
# src/data_generator/main.py

AUGMENTATION_PIPELINE = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(degrees=20),
    transforms.RandomResizedCrop(224, scale=(0.7, 1.0)),
    transforms.ColorJitter(
        brightness=0.3,
        contrast=0.3,
        saturation=0.3,
        hue=0.1
    ),
])
```

```

Augmented files are prefixed with `aug\_` to prevent re-augmentation on subsequent runs.

#### ### CLIP Relevance Filtering

When enabled, the `CLIPRelevanceVerifier` (`src/filters/clip\_relevance.py`) loads the `openai/clip-vit-base-patch32` model and computes cosine similarity between each image's CLIP embedding and a text probe constructed from the class label and user prompt (via `make\_text\_probe()` in `src/filters/text\_probes.py`). Images scoring below `RELEVANCE\_THRESHOLD` (default 0.22) are deleted from disk. The filter is applied to both `train/` and `test/` subdirectories for each class. A minimum post-filter image count (`MIN\_IMAGES\_AFTER\_FILTER`, default 3) is enforced; if any class falls below this threshold, the pipeline returns an error rather than training on an insufficient dataset.

#### ### Corrupt Image Removal

During data loading, the `Trainer.remove\_corrupt\_images()` method attempts to open and fully load every image with Pillow. Files that fail are deleted from disk and excluded from the dataset.

#### ### Normalization for Model Ingestion

At training and evaluation time, all images are resized to 224×224 pixels and normalized to ImageNet channel statistics. The preprocessing transform chain is:

```

```python

```

```
# src/trainer/train.py
```

```
data_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])
````
```

### ## Storage and Formatting

#### ### Local Filesystem

Images are stored on disk under `downloads/<class\_name>/` (post-scraping) and reorganized into `downloads/train/<class>/` and `downloads/test/<class>/` after splitting. Filenames include a zero-padded index and an 8-character MD5 hash of the source URL for deduplication. The `LocalStorage` class (`src/scrapper/storage.py`) handles streaming downloads with configurable timeouts.

#### ### SQLite Metadata Store

An optional SQLite database (`image\_scraper.db`) persists lightweight metadata per image. The schema enforces a `UNIQUE(keyword, image\_url)` constraint with `INSERT OR IGNORE` semantics to handle duplicate URLs gracefully. Image binary data is never stored in the database. The schema is as follows:

| Column       | Type    | Constraint                |
|--------------|---------|---------------------------|
| ---          | ---     | ---                       |
| `id`         | INTEGER | PRIMARY KEY AUTOINCREMENT |
| `keyword`    | TEXT    | NOT NULL                  |
| `image_url`  | TEXT    | NOT NULL                  |
| `local_path` | TEXT    | —                         |
| `scraped_at` | TEXT    | NOT NULL                  |

\*Table 1. SQLite images table schema.\*

#### ### PyTorch Dataset

For training, `torchvision.datasets.ImageFolder` is used to infer class labels from the directory hierarchy under `downloads/train/`. The trainer performs label remapping to ensure contiguous 0-based indices compatible with `nn.CrossEntropyLoss`, filtering samples to include only the classes specified in the original user prompt.

---

## # Methodology

### ## System Architecture

The system follows a three-tier architecture: a React single-page application (frontend), a FastAPI server (backend orchestrator), and external inference APIs (Groq for LLM, Hugging Face for diffusion).

The end-to-end pipeline executes the following stages sequentially within a single synchronous `POST /run/` request handler:

1. **Prompt Parsing** — The user's natural-language prompt is forwarded to the Groq-hosted LLM, which returns structured JSON containing class names and per-class search queries.
2. **Data Collection** — For each class-keyword pair, the scraper launches a headless browser, retrieves image URLs from Bing, and downloads them to local disk.
3. **Synthetic Generation (optional)** — The LLM generates diffusion-optimized prompt variations, which are sent to the FLUX.1-schnell model via the Hugging Face Inference API. Generated images are saved alongside scraped images.
4. **Dataset Assembly** — The DataGenerator splits images into train/test sets (80/20).
5. **CLIP Filtering (optional)** — Low-relevance images are purged from both splits based on CLIP cosine similarity scores.
6. **Augmentation (optional)** — Training images are augmented with random geometric and photometric transforms.
7. **Fine-Tuning** — A pretrained CNN backbone is loaded, its feature-extraction layers are frozen, and a new classification head is trained on the assembled dataset.
8. **Evaluation** — Triggered separately via `POST /test/`, which loads the finetuned checkpoint and a fresh pretrained baseline, runs inference on the held-out test set, and returns confusion matrices and per-class accuracy metrics for both.

### ## Algorithmic Choices

#### ### Transfer Learning with Frozen Backbones

The project adopts a transfer-learning paradigm: a CNN pretrained on ImageNet-1K is loaded, all backbone parameters are frozen (`requires\_grad = False`), and only a newly initialized classification head (a single `nn.Linear` layer matching the number of target classes) is trained. This design choice is motivated by the small, noisy datasets typical of automated scraping and generation — freezing the backbone prevents catastrophic forgetting and reduces training time to minutes rather than hours.

### ### CLIP-Based Quality Assurance

Rather than relying solely on keyword relevance from the search engine, the system employs CLIP (`openai/clip-vit-base-patch32`) as a semantic filter. By computing image–text cosine similarity against a probe derived from the class label and the user's task description, the system removes off-topic or low-quality images before they enter the training set. The text probe is constructed as:

```
```python
# src/filters/text_probes.py

def make_text_probe(class_name, user_prompt=None):
    if user_prompt:
        return f'{user_prompt}. A clear photo showing: {class_name}.'
    return f'a photo of {class_name}'
```
```

### ### LLM-as-Orchestrator Pattern

The LLM serves a dual role: (a) as a structured information extractor (parsing natural language into class labels and search queries) and (b) as a prompt engineer (generating diverse, high-quality diffusion prompts). The `json\_repair` library provides resilience against malformed JSON output from the LLM, a common failure mode with instruction-following models.

### ### Headless Browser Scraping

Playwright was chosen over static HTTP scraping (e.g., `requests` + BeautifulSoup alone) because Bing Image Search renders results via JavaScript, making server-side HTML insufficient. The scraper performs three scroll iterations to trigger lazy-loading of additional image tiles.

### ## Integration

### Backend API (FastAPI)

The backend exposes four REST endpoints:

| Endpoint                         | Method | Purpose                                                  |
|----------------------------------|--------|----------------------------------------------------------|
| --- --- ---                      |        |                                                          |
| <code>/run/</code>               | POST   | Executes the full data-generation and training pipeline  |
| <code>/test/</code>              | POST   | Runs baseline vs. finetuned evaluation on the test split |
| <code>/parse-user-prompt/</code> | POST   | Standalone prompt-parsing (LLM call only)                |
| <code>/generate-dataset/</code>  | POST   | Standalone scraping by keyword                           |

\*Table 2. FastAPI REST endpoint inventory.\*

Request/response bodies use Pydantic models (`RunRequest`, `DataSources`, `AdvancedSettings`) for validation. CORS is configured to allow requests from `localhost:5173` and `localhost:3000` (Vite and alternative dev server ports).

### Frontend (React + Vite + Tailwind CSS)

The SPA communicates with the backend via `fetch()` calls in `src/frontend/src/utils/api.ts`. The primary interaction flow is:

1. The user fills out a `PromptForm` component (prompt, email, model selection, data source toggles, advanced hyperparameters).
2. On submission, a JSON payload is POSTed to `/run/`. The request is synchronous and blocking — the frontend displays a timed pipeline progress visualization (`PipelineStepper`) driven by estimated per-stage durations while waiting for the backend response.
3. After `/run/` returns, the frontend transitions to a polling phase, sending `POST /test/` requests every 5 seconds (with a configurable 75-second timeout) until evaluation metrics are available.
4. Results (confusion matrices, accuracy delta) are persisted in `localStorage` and rendered on a dedicated Results page.

Run state (phase, timestamps, errors, data-source configuration) is synchronized across pages via `localStorage` and a custom `runstatechange` DOM event, consumed by the `useRunState()` hook.

### External API Integrations

**\*\*Groq API:\*\*** The ``CustomLLM`` class (``src/utils/llm.py``) wraps a standard OpenAI-compatible chat completions endpoint. Authentication is via a bearer token (``GROQ_API_KEY``). The ``trust_env`` flag on the ``requests.Session`` is explicitly disabled to avoid proxy interference.

**\*\*Hugging Face Inference API:\*\*** The ``InferenceClient`` from ``huggingface_hub`` is used with the ``hf-inference`` provider for text-to-image generation. Authentication is via ``HF_TOKEN``.

---

# Models

## Architecture and Selection

The system employs models across four distinct functional roles:

### Large Language Model (Prompt Parsing and Prompt Engineering)

| Attribute | Value                                                                                                                |
|-----------|----------------------------------------------------------------------------------------------------------------------|
| Model     | <code>`openai/gpt-oss-120b`</code> (120B-parameter open-source model)                                                |
| Provider  | Groq inference API                                                                                                   |
| Usage     | Extracting class labels and search queries; generating base diffusion prompts; producing up to 100 prompt variations |

\*Table 3. LLM specification.\*

### Text-to-Image Diffusion Model (Synthetic Data Generation)

| Attribute              | Value                                                                                                                                                             |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Primary Model          | <code>`black-forest-labs/FLUX.1-schnell`</code> (fast distilled variant)                                                                                          |
| Provider               | Hugging Face Inference API (serverless)                                                                                                                           |
| Explored Alternative 1 | <code>`stable-diffusion-v1-5`</code> — local inference via <code>`diffusers`</code> <code>`AutoPipelineForText2Image`</code> , FP16, <code>`torch.compile`</code> |
| Explored Alternative 2 | <code>`stable-diffusion-xl-base-1.0`</code> via <code>`StableDiffusionAPI`</code> REST endpoint                                                                   |
| Explored Alternative 3 | <code>`FLUX.1-dev`</code> via Gradio client                                                                                                                       |

\*Table 4. Diffusion model specification and explored alternatives.\*

### ### CLIP Model (Relevance Filtering)

| Attribute | Value                                                                                   |
|-----------|-----------------------------------------------------------------------------------------|
|           |                                                                                         |
| Model     | `openai/clip-vit-base-patch32` (ViT-B/32)                                               |
| Provider  | Loaded locally via `transformers.CLIPModel` and `CLIPProcessor`                         |
| Usage     | Cosine similarity between image and text embeddings to filter irrelevant scraped images |
|           |                                                                                         |

\*Table 5. CLIP model specification.\*

### ### Vision Classification Models (Fine-Tuning Targets)

The user selects from four ImageNet-1K pretrained architectures, all loaded via `torchvision.models`:

| Enum Value     | Architecture      | Pretrained Weights                         |
|----------------|-------------------|--------------------------------------------|
|                |                   |                                            |
| `RESNET_50`    | ResNet-50         | `ResNet50_Weights.IMAGENET1K_V1`           |
| `MOBILENET_V3` | MobileNetV3-Large |                                            |
|                |                   | `MobileNet_V3_Large_Weights.IMAGENET1K_V1` |
| `EFFICIENTNET` | EfficientNet-B0   | `EfficientNet_B0_Weights.IMAGENET1K_V1`    |
| `CONV_NEXT`    | ConvNeXt-Tiny     | `ConvNeXt_Tiny_Weights.IMAGENET1K_V1`      |

\*Table 6. Supported vision model architectures and their pretrained weight identifiers.\*

## ## Implementation Details

### ### LLM Access

The `CustomLLM` wrapper (`src/utils/llm.py`) constructs a standard OpenAI-compatible chat completions payload and sends it to the configured Groq endpoint. No fine-tuning or local model hosting is performed — all LLM inference is API-based.

### ### Diffusion Model Access

FLUX.1-schnell is accessed exclusively via the Hugging Face serverless Inference API (`InferenceClient.text\_to\_image`). No local diffusion model weights are downloaded or fine-tuned. The alternative Stable Diffusion v1.5 path in `text\_to\_image.py` uses `AutoPipelineForText2Image.from\_pretrained()` with FP16 precision and

``torch.compile(mode="reduce-overhead", fullgraph=True)`` for UNet acceleration, but this path is not invoked by the main pipeline.

### ### CLIP Model Access

The CLIP model is loaded locally from Hugging Face Hub on first use (lazy initialization) and cached in memory for subsequent scoring calls within the same process lifetime. Inference runs in ``torch.no_grad()`` mode with explicit L2-normalization of both image and text embeddings before computing dot-product similarity.

### ### Vision Model Fine-Tuning

The ``ModelBuilder`` class (``src/utils/model_builder.py``) implements a transfer-learning strategy:

1. Load a pretrained backbone from ``torchvision.models`` with official ImageNet-1K weights.
2. Freeze all parameters: ``param.requires_grad = False`` for every parameter.
3. Replace the final classification head via ``_replace_classifier_head()``, which locates the terminal ``nn.Linear`` layer (``fc`` for ResNet, or the last ``nn.Linear`` found via ``named_modules()`` for other architectures) and replaces it with a new ``nn.Linear(in_features, num_classes)``. The new head's parameters are trainable by default.
4. Move the model to GPU if available.

The trained checkpoint is saved as a dictionary containing both the ``state_dict`` and the architecture identifier string, enabling the evaluation module to reconstruct the correct skeleton at load time. The classifier head replacement logic is:

```
```python
# src/utils/model_builder.py

for param in model.parameters():
    param.requires_grad = False

# For ResNet family:
in_features = model.fc.in_features
model.fc = nn.Linear(in_features, num_classes)

# For other architectures: find last nn.Linear via named_modules()
# and replace with nn.Linear(in_features, num_classes)
```
```

## ## Parameters and Prompting

### ### Training Hyperparameters

| Parameter                  | Default Value          | Source                       |
|----------------------------|------------------------|------------------------------|
| Optimizer                  | Adam                   | `src/trainer/train.py`       |
| Learning rate              | 0.003                  | `src/trainer/train.py`       |
| Loss function              | `nn.CrossEntropyLoss`  | `src/trainer/train.py`       |
| Batch size (training)      | 50                     | `src/trainer/train.py`       |
| Batch size (evaluation)    | 32                     | `src/trainer/test.py`        |
| Epochs                     | 12 (user-configurable) | `src/utls/models.py`         |
| Input resolution           | $224 \times 224$       | `src/trainer/train.py`       |
| Train/test split           | 80 / 20                | `src/data_generator/main.py` |
| Augmented copies per image | 5                      | `src/data_generator/main.py` |
| Random seed (split)        | 42                     | `src/data_generator/main.py` |

\*Table 7. Training hyperparameters with default values and source locations.\*

### ### CLIP Filtering Parameters

| Parameter                   | Default Value                  | Source          |
|-----------------------------|--------------------------------|-----------------|
| CLIP model                  | `openai/clip-vit-base-patch32` | `src/config.py` |
| Relevance threshold         | 0.22                           | `src/config.py` |
| Minimum images after filter | 3                              | `src/config.py` |

\*Table 8. CLIP relevance filtering configuration defaults.\*

### ### Explored Diffusion Parameters

The following diffusion-related parameters were explored in experimental scripts but are not used in the main production pipeline:

| Parameter                | Value                       | Source                    |
|--------------------------|-----------------------------|---------------------------|
| Guidance scale sweep     | 0.0 through 10.0 (step 1.0) | `text_to_image.py`        |
| Precision                | FP16 (`torch.float16`)      | `text_to_image.py`        |
| Inference steps (SD API) | 30                          | `stable_diffusion_api.py` |
| Guidance scale (SD API)  | 7.5                         | `stable_diffusion_api.py` |

| Resolution (SD API) |  $512 \times 512$  | `stable\_diffusion\_api.py` |  
| Negative prompt | Defined to suppress common artifacts | `stable\_diffusion\_api.py` |

\*Table 9. Explored diffusion model parameters from experimental scripts.\*